Outil de visualisation de plans d'expériences

# Dim2Dir : lire un plan d'expériences en deux dimensions

Philippe AMUZUGA Dr.-Ing.

Documentation et guide d'utilisation

Édition 2026

Mettre à plat les **facteurs** pour voir quelles combinaisons ont été **essayées**.

Version 2.1.0 : novembre 2026

# Sommaire

Quatre parties et une conclusion


1. **Comprendre** : relier un tableau d'essais à une grille emboîtée.
2. **Fonctionnement** : suivre les niveaux, les positions et les couleurs.
3. **S'en servir en ligne de commande** : lancer et vérifier les sorties.
4. **Dans un programme Python** : visualiser et personnaliser un tableau.
5. **Conclusion** : retrouver les sources, les tests et la licence.


**Objectif**

Produire une visualisation depuis Excel ou pandas, puis expliquer ses cases, personnaliser son rendu et choisir ses sorties.

# Plusieurs facteurs, une feuille

Comment regarder un plan à plusieurs facteurs sur une feuille de papier ?

# Partie 1

## Comprendre

Lire une grille

# Un tableau conserve les quatre facteurs du tournage L9

Un facteur est un réglage, un niveau une valeur possible ; chaque ligne est un essai.

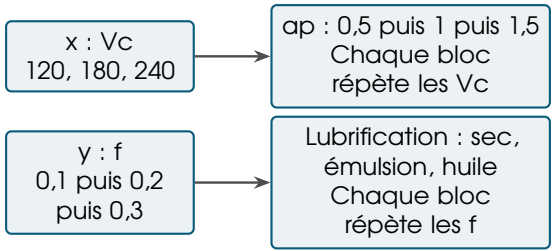
| Run | Vc m/min | f mm/tr | ap mm | Lubrification |
|---|---|---|---|---|
| 1 | 120 | 0,1 | 0,5 | sec |
| 2 | 120 | 0,2 | 1,0 | émulsion |
| 3 | 120 | 0,3 | 1,5 | huile |

Extrait : trois des neuf essais du L9 ; tableau complet dans `examples/tournage_L9.xlsx`. Vc : vitesse de coupe ; f : avance par tour ; ap : profondeur de passe.

Le dessin doit conserver les quatre réglages de chaque essai.

# Deux axes emboîtés conservent tous les facteurs

Par défaut, x reçoit les facteurs impairs et y les facteurs pairs, après exclusion de `Run`. Le premier facteur de chaque axe varie le plus vite.

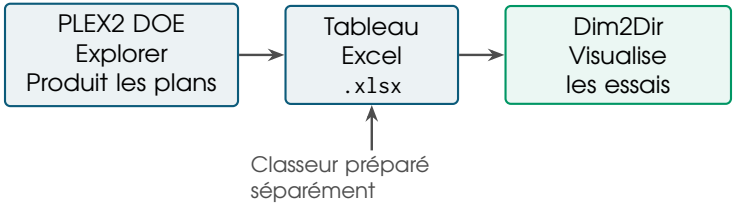



**Définition : un axe emboîté**

Lire une position revient à retrouver un niveau dans chaque bloc. Ici, chaque direction contient $3 \times 3 = 9$ positions.

# Dim2Dir visualise les plans de PLEX2 ou un fichier autonome

PLEX2 DOE Explorer produit les plans Excel [1]. Dim2Dir lit un tableau existant et ajoute sa représentation. On peut aussi partir de son propre classeur.




Le fichier source reste intact. openpyxl lit et écrit le classeur [2]. La chaîne de traitement calcule les niveaux et les positions, puis les couleurs et les exports.

# Ce qu'il faut retenir

Comprendre

**Points clés**

- Une ligne est un essai, une colonne un facteur.
- Les niveaux s'emboîtent : le premier facteur de l'axe varie le plus vite.
- Couverture : part des cases occupées (L9 : 9 sur 81, soit 11,1 %).
- La couverture porte sur les niveaux observés dans le tableau ; elle ne mesure ni l'optimalité ni la qualité statistique du plan.

PARTIE 2

# Fonctionnement

Des niveaux aux cases

# Chaque facteur devient une liste de niveaux observés

`_build_visual_axis` retire les valeurs vides et les doublons. Si tous les niveaux d'un facteur sont numériques, ils sont triés ; sinon ils gardent leur ordre d'apparition. Les libellés sont tronqués à 36 caractères.

```
    unique_values = _unique_nonblank_values(values)
    if _all_numeric(unique_values):
        unique_values = sorted(unique_values, key=lambda value: float(value))
```

| Visualisation - Feuil1 | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| Traitement thermique: revenu | E7 | E16 | E25 | E8 | E17 | E26 | E9 | E18 | E27 |
| Traitement thermique: trempe | E4 | E13 | E22 | E5 | E14 | E23 | E6 | E15 | E24 |
| Traitement thermique: aucun | E1 | E10 | E19 | E2 | E11 | E20 | E3 | E12 | E21 |
| Matériau | acier S355 | uminium 606 | titane TA6V | acier S355 | uminium 606 | titane TA6V | acier S355 | uminium 606 | titane TA6V |
| Épaisseur mm | 2 | | | 4 | | | 6 | | |

Acier S355, aluminium 6061, titane TA6V : cet ordre vient du tableau. Chaque catégorie occupe une position entière. Ici toutes les cases sont remplies : c'est normal pour un plan factoriel complet ($3 \times 3 \times 3$ essais).

Sortie du programme sur `materiaux_textuels.xlsx`, données synthétiques.

# Les pas et les indices placent chaque essai dans la grille

Un pas indique combien de cases franchir pour changer de niveau. `_axis_strides` multiplie les nombres de niveaux précédents. Les indices commencent à zéro ; le premier facteur de chaque axe varie le plus vite.

```
x_axes = axes[0::2]
y_axes = axes[1::2]
```

$$s_1 = 1, \qquad s_j = \prod_{i<j} n_i, \qquad x = \sum_j i_{x,j} s_{x,j}, \qquad y = \sum_j i_{y,j} s_{y,j}.$$

**Exemple L9 : pas $(1, 3)$ sur chaque axe**

x porte Vc et ap, y porte f et Lubrification. La ligne vaut $H - 1 - y$, avec $H = 9$.
E1 : indices $(0, 0, 0, 0)$, position $(x, y) = (0, 0)$, ligne 8.
E2 : indices $(0, 1, 1, 1)$, position $(3, 4)$, ligne 4.

# Le rang colore les cases, les collisions réunissent les essais

Par défaut, le premier essai est clair et le dernier atteint la couleur de base. La couleur suit l'ordre des lignes, sans représenter une réponse mesurée.

```python
def _trial_sequence_color(base_color: str, order: int, n_runs: int) -> str:
    ratio = 1.0 if n_runs <= 1 else (order - 1) / (n_runs - 1)
    return _blend_hex_colors("FFFFFF", base_color, 0.22 + (0.78 * ratio))
```

| Visualisation - Feuil1 | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| f mm/tr: 0.3 | | | | | | | | | E4 |
| f mm/tr: 0.2 | E5 | | | | E3 | | | | |
| f mm/tr: 0.1 | E1<br>E2 | | | | E6 | | | | |
| Vc m/min | 120 | 180 | 240 | 120 | 180 | 240 | 120 | 180 | 240 |
| Lubrification | sec | | | huile | | | émulsion | | |

Données synthétiques : `plan_avec_collisions.xlsx`.

E1 et E2 partagent la même case ; sa couleur est celle du premier essai qui l'occupe. Une case blanche signale une combinaison non essayée.

**Piège : donnée manquante**

Vc manque pour E5 : indice zéro, ici 120, avec avertissement. Vérifier le tableau.

# Ce qu'il faut retenir

Comment ça marche et garde-fous

**Points clés**

- Les indices de niveaux et leurs pas déterminent la case.
- La couleur suit l'ordre des lignes ; une collision réunit les identifiants.

**Piège : la grille croît par multiplication**

Plafond par défaut : 90 000 cases. Neuf facteurs à quatre niveaux en donnent $4^9 = 262\,144$. Au-delà : statut `skipped`, aucune image, message rouge dans le classeur si une sortie XLSX est demandée. Relever `--max-cells` ou réduire les facteurs. `--dry-run` n'écrit rien (ni classeur, ni rapport, ni image).

# Passer à son classeur

Et en pratique, comment obtenir puis vérifier cette image ?

# Partie 3

## En terminal

Du classeur à la grille

# Deux façons d'utiliser Dim2Dir : un fichier ou pip

Python 3.10 ou plus ; même code, même API. Classeur : en-têtes contigus en ligne 1, essais dès la ligne 2.

**Fichier seul : installer d'abord ses dépendances**

```
python -m pip install "openpyxl>=3.1" "matplotlib>=3.7" "pillow>=10"
```

**Télécharger `Dim2Dir.py`**

```
python Dim2Dir.py plan.xlsx
```

**Installer le paquet**

```
pip install dim2dir
dim2dir plan.xlsx Feuil1 --output-dir resultats
```

Feuille par défaut de la ligne de commande : `Feuil1` si elle existe, sinon la première. L'API prend la première.

Extras : `[pandas]` ajoute les DataFrames (tableaux pandas), `[excel]` exige Windows et Excel.

# Les options choisissent les sorties et personnalisent le rendu

| Sélection et sorties | Effet |
|---|---|
| `excel_file`, `sheet_name` | Source et feuille |
| `--all-sheets`, `--list-sheets` | Traiter tout, lister |
| `--output-dir` | Dossier de sortie |
| `--outputs xlsx,png,pdf,svg` | Formats demandés |
| `--image-engine`, `--dpi` | Moteur, dpi (points par pouce) |
| `--x-factors`, `--y-factors` | Partition des facteurs |
| `--report`, `--dry-run` | JSON, sans écriture |
| `--max-cells`, `--max-pixels` | Plafonds grille, image |
| `--max-runs`, `--excel-timeout` | Essais, délai Excel |

| Personnalisation | Effet |
|---|---|
| `--style fichier.json` | Charger un Style |
| `--palette`, `--palette-reverse` | Palette (gamme de couleurs), inversion |
| `--color`, `--font-color` | Base, texte |
| `--font`, `--font-size` | Police, points |
| `--cell-size`, `--label-scale` | Cellule, étiquettes |
| `--line-color`, `--line-style` | Bordures |
| `--title`, `--no-title` | Titre, suppression |
| `--legend`, `--label-format` | Légende, format y |
| `--run-prefix`, `--level-order` | Préfixe, niveaux |

Les options explicites priment sur le JSON. `--outputs` remplace `--no-xlsx`, `--no-png` et `--no-pdf`. L'aide `--help` détaille les valeurs acceptées.

# Les trois sorties usuelles se complètent d'un SVG

| Visualisation - Feuil1 | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Lubrification: huile | f mm/tr: 0.3 | | | | | | | E3 | | |
| | f mm/tr: 0.2 | | | E8 | | | | | | |
| | f mm/tr: 0.1 | | | | | E4 | | | | |
| Lubrification: émulsion | f mm/tr: 0.3 | | E6 | | | | | | | |
| | f mm/tr: 0.2 | | | | E2 | | | | | |
| | f mm/tr: 0.1 | | | | | | | | | E7 |
| Lubrification: sec | f mm/tr: 0.3 | | | | | | E9 | | | |
| | f mm/tr: 0.2 | | | | | | | | E5 | |
| | f mm/tr: 0.1 | E1 | | | | | | | | |
| Vc m/min | | 120 | 180 | 240 | 120 | 180 | 240 | 120 | 180 | 240 |
| ap mm | | 0.5 | | | 1 | | | 1.5 | | |

| | |
|---|---|
| XLSX | Tableau et dessin |
| PDF | Figure vectorielle |
| PNG | Image selon les dpi |
| SVG | Vectoriel, sur demande |

Neuf cases occupées sur 81 possibles : couverture de 11,1 %.

**Intégrité du classeur**

La source reste intacte. Dans la copie, openpyxl peut perdre images, graphiques et formes. La zone à droite du tableau doit être libre.

Pour `plan.xlsx`, feuille `Feuil1` : `plan_visu.xlsx`, `plan_visu_Feuil1.png` et `.pdf` (SVG sur demande). Exemple pandas : `resultats_pandas/plan_visu_Feuil1.png`. Figure : `tournage_L9.xlsx`, données synthétiques.

# Changer les axes réorganise le même factoriel de soudage

Le plan $2^4$ (factoriel complet : quatre facteurs à deux niveaux) compte seize essais synthétiques, qui occupent toute la grille $4 \times 4$. L'affectation des axes change les voisinages, pas les combinaisons.

**Défaut : x = I, Gaz**

| Visualisation - Feuil1 | | | | | |
|---|---|---|---|---|---|
| Tension V: 26 | Avance mm/s: 6 | E6 | E14 | E8 | E16 |
| | Avance mm/s: 4 | E2 | E10 | E4 | E12 |
| Tension V: 22 | Avance mm/s: 6 | E5 | E13 | E7 | E15 |
| | Avance mm/s: 4 | E1 | E9 | E3 | E11 |
| I A | | 140 | 180 | 140 | 180 |
| Gaz L/min | | 12 | | 16 | |

**Variante : x = I, Avance**

| Visualisation - Feuil1 | | | | | |
|---|---|---|---|---|---|
| Tension V: 26 | Gaz L/min: 16 | E4 | E12 | E8 | E16 |
| | Gaz L/min: 12 | E2 | E10 | E6 | E14 |
| Tension V: 22 | Gaz L/min: 16 | E3 | E11 | E7 | E15 |
| | Gaz L/min: 12 | E1 | E9 | E5 | E13 |
| I A | | 140 | 180 | 140 | 180 |
| Avance mm/s | | 4 | | 6 | |

```
--x-factors "I A,Avance mm/s"
--y-factors "Gaz L/min,Tension V"
```

Sorties du même `soudage_2p4.xlsx`. À gauche, y porte Avance et Tension. À droite, y porte Gaz et Tension.

# Ce qu'il faut retenir

Dépannage et bonnes pratiques

**Points clés**

- Sortie ouverte dans Excel : la fermer avant de relancer.
- Feuille introuvable : le message liste les feuilles.
- Formule sans cache : recalculer et enregistrer le classeur dans Excel.
- Lire le résumé et les avertissements avant d'interpréter la grille.
- Essayer d'abord avec `--dry-run`, qui n'écrit aucun fichier.

La simulation vérifie le tableau et sa couverture. Contrôler ensuite la grille exportée, surtout après un changement d'axes ou de limites.

# Intégrer la visualisation

Et dans votre propre programme Python ?

# Partie 4

## Dans Python

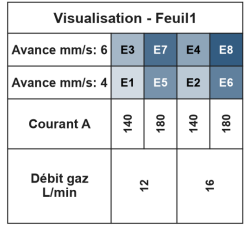
Du tableau à l'application

# Projeter, exporter ou obtenir une figure depuis un DataFrame

`df` est un DataFrame pandas [3] : voir `examples/client_project/depuis_pandas.py` (huit essais de soudage synthétiques). Il n'est pas modifié.

```
import dim2dir
projection = dim2dir.project(df)
result = dim2dir.visualize(
    df, output_dir="resultats_pandas",
    outputs=("png", "pdf"))
print(result.files, result.warnings)
```

`projection.occupied`, `.width` et `.height` donnent la couverture sans exporter.




`resultats_pandas/plan_visu_Feuil1.png`

Dans un notebook, terminer la cellule par `fig`. L'appel retourne une figure matplotlib [4], sans affichage automatique.

```
fig = dim2dir.figure(df)
fig
```

# Un objet Style règle palette, police et tailles

| Champ | Défaut | Effet |
|---|---|---|
| `palette` | `None` | Dégradé d'origine A1, sinon 335C81 |
| `base_color` | `None` | Inverser les couleurs |
| `palette_reverse` | `False` | Arial |
| `font_family` | `None` | Essais 9, niveaux de x 9, étiquettes de y et noms d'axes 10, titre 11 (en points) |
| `font_size` | `None` | Axes/titre : 1F1F1F |
| `font_color` | `None` | Espace des axes |
| `label_scale` | `1.0` | Contraste auto. |
| `cell_font_color` | `None` | |

| Champ | Défaut | Effet |
|---|---|---|
| `cell_size` | `34` | Pixels à 96 dpi |
| `line_color` | `1F1F1F` | Bordures |
| `line_style` | `thin` | Épaisseur |
| `title` | `None` | Titre automatique |
| `legend` | `False` | Légende des rangs |
| `label_format` | `{name}: {level}` | Étiquettes y |
| `run_prefix` | `E` | Préfixe des essais |
| `level_orders` | `None` | Ordre des niveaux |

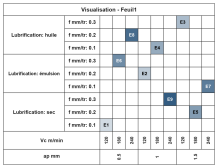
`Style` est immuable et valide ses seize champs. `None` active le comportement d'origine. Avec une palette, la couleur de base est ignorée. Un titre vide retire son texte.

Les valeurs par défaut reproduisent le rendu d'origine.

# Trois rendus d'un même plan de tournage

Le L9 conserve ses neuf essais synthétiques et leurs positions. Les paramètres changent uniquement l'affichage. Les étiquettes identifient les essais dans chaque rendu. La couverture reste identique.

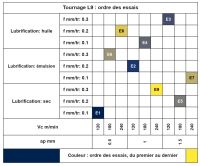
## Défaut




```
Style()
```

Dégradé blanc vers bleu, sans légende supplémentaire. Le tableau et les axes restent identiques.

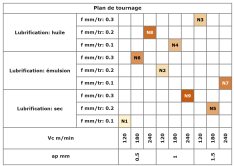
## Cividis




```
Style(
  palette="cividis",
  legend=True,
  title="Tournage L9 : "
        "ordre des essais")
```

Palette lisible par les daltoniens ; la légende explique le rang.

## Personnalisé




```
Style(
  palette=("FFF2CC","C55A11"),
  font_family="Verdana",
  font_size=11,cell_size=40,
  line_color="7F7F7F",
  line_style="medium",
  run_prefix="N",
  title="Plan de tournage",
  label_scale=1.3)
```

# Choisir la sortie et connaître les limites

Sans `outputs` : un classeur donne XLSX, PNG et PDF ; un DataFrame donne seulement PNG. Sans `output_dir`, un DataFrame retourne les octets (`Result.data["png"]`) ; sinon `Result.files` liste les chemins. `Result.report` résume en texte ; `report=True` ajoute un JSON.

| Champ de Limits | Défaut | Dépassement |
|---|---|---|
| `max_cells` | 90 000 | `visualize` : `skipped` |
| `max_pixels` | 60 000 000 | `visualize` : `skipped` |
| `max_runs` | 20 000 | `Dim2DirError` |
| `excel_timeout_s` | 120 secondes | Erreur Excel ou repli en `auto` |

Relever `max_cells` ne supprime pas les limites d'Excel (16 384 colonnes, 1 048 576 lignes). `max_pixels` protège aussi PDF et SVG. `Result` n'a pas d'attribut de statut : `skipped` figure dans les rapports.

# Ce qu'il faut retenir

Dans un programme Python

**Points clés**

- `project` calcule la couverture sans produire de fichier.
- `figure` retourne une figure pour le notebook ou l'application.
- `visualize` livre les formats demandés, sur disque ou en mémoire.
- `Style` personnalise le rendu sans changer les combinaisons essayées.
- `Limits` protège les ressources ; lire les avertissements retournés.

Le même appel traite un classeur ou un DataFrame. Préparer les facteurs, choisir la sortie et vérifier la couverture avant d'intégrer l'image à une application.

# Conclusion

## Sources et tests

Une source, deux façons de l'utiliser

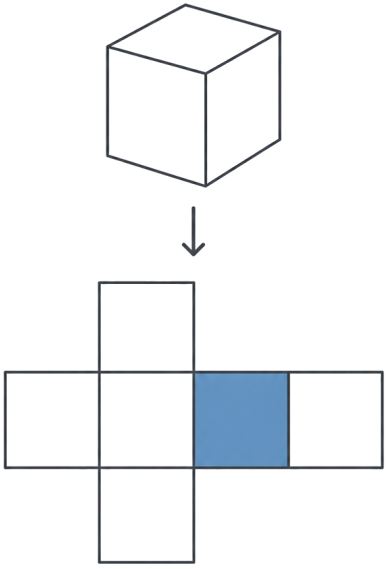
# Ce qu'il faut retenir

Pour les développeurs : structure, tests, licence

**Points clés**

- Source unique : `src/dim2dir/core.py`, commune au paquet et au fichier autonome.
- `tools/sync_standalone.py` synchronise la copie autonome `Dim2Dir.py`.
- Tests : `pytest`, API, limites, exports et parité avec la version 1.
- Licence : GNU AGPLv3, avec possibilité d'utiliser une version ultérieure.
- Projet parent : dépôt PLEX2 DOE Explorer [1].

Modifier la source unique, synchroniser la copie, puis exécuter les tests.



**Merci de votre attention**

Questions ?

# Références


[1] Philippe Amuzuga. PLEX2 DOE Explorer. https://github.com/Philippe-AMUZUGA/plex2-doe-explorer.

[2] Équipe openpyxl. Documentation d'openpyxl. https://openpyxl.readthedocs.io.

[3] Équipe pandas. pandas : analyse de données en python. https://pandas.pydata.org.

[4] Équipe Matplotlib. Matplotlib : visualisation en python. https://matplotlib.org.

Outil de visualisation de plans d'expériences

# Dim2Dir : lire un plan d'expériences en deux dimensions

Philippe AMUZUGA Dr.-Ing.

Documentation et guide d'utilisation

Édition 2026

Mettre à plat les **facteurs** pour voir quelles combinaisons ont été **essayées**.

Version 2.1.0 : novembre 2026